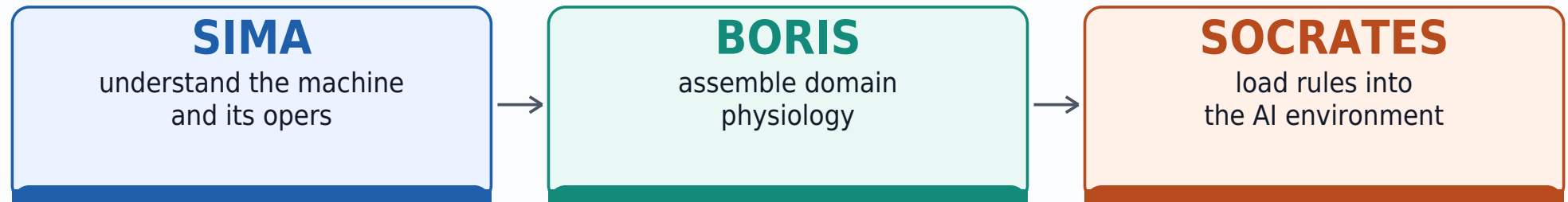


SOCRATES for AI

through BOIS: SIMA & BORIS

How to program AI products with physiology, not just with a prompt



Updated educational article: now with SIMA&BORIS, SOCRATES-programming practice, and examples for AI products.

1. Why this article is needed

The first version explained SOCRATES/SOKRAT as an approach to organizing calculation, but it was too general. It lacked the key practical layer: how BOIS is used through SIMA&BORIS and how this becomes programming for AI products.

! Main idea

SOCRATES/SOKRAT is not a "good prompt". It is a way to describe the physiology of calculation: domain, values, substrate, rules, state, organs, stops, procedures, tests, and feedback. The AI executes not a role, but a loaded map of permitted transitions.

In simpler words: a normal prompt asks the AI to "be helpful". A SOCRATES program explains which states the AI must distinguish, which transitions are allowed, what counts as evidence, where to stop, who owns the decision, and how the result must be checked.

From instruction to physiology



The article then shows the complete route: BOIS supplies the grammar; SIMA reconstructs the machines already acting; BORIS assembles a domain physiology; and SOCRATES/SOKRAT turns that physiology into an executable package for AI.

2. Mini-glossary without academic fog

These words are not used for terminology itself. They keep different operations distinct.

Term	Plain meaning	Why it matters in an AI product
BOIS	A grammar of organized activity with consequences under incomplete knowledge.	Prevents AI from presenting a guess as fact; requires D/V/S, boundaries, and checking.
SIMA [substrate-independent machine analyzer]	The analyst: looks at existing behavior and reconstructs what machine is operating there.	Helps understand users, processes, risks, error sources, and real ops.
BORIS [rational operational-intelligence system of business]	The implementer: turns BOIS into a narrow working physiology for a domain.	Creates product rules: roles, stops, state, procedures, tests, and answer surface.
SOCRATES/SOKRAT	Engineering rule-based programming of a computing environment.	Loads into AI not a "personality", but a map of calculation and execution.
Calculation	A reproducible state transition through operation, cost, memory, and feedback.	An AI answer should be not a stream of text, but the closure of a calculated transition.
UCE [universal computing environment]	An environment able to execute different architectures of inquiry and action.	LLM, tools, files, memory, people, and procedures together.

i

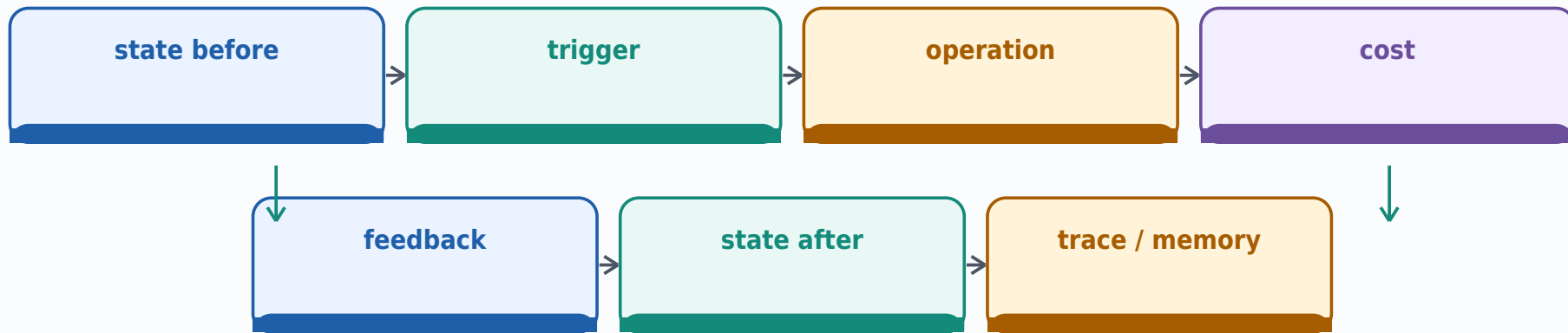
Important

SIMA does not issue a hidden diagnosis, BORIS is not a second BOIS core, and SOCRATES/SOKRAT does not prove truth. These are engineering roles inside a process.

3. What it means to "organize calculation"

In everyday language, calculation often sounds like arithmetic. In BOIS it is broader: a reproducible transformation of distinguishable states. There was one state; a trigger arrived; an operation was performed; a cost, trace, change, and feedback appeared.

calculation = transition



For an AI product, this means every answer must have a place in a transition. AI does not simply "say text"; it distinguishes the initial state, checks the domain, chooses an allowed protocol, marks the unknown, proposes an action, and shows what will change after the action.

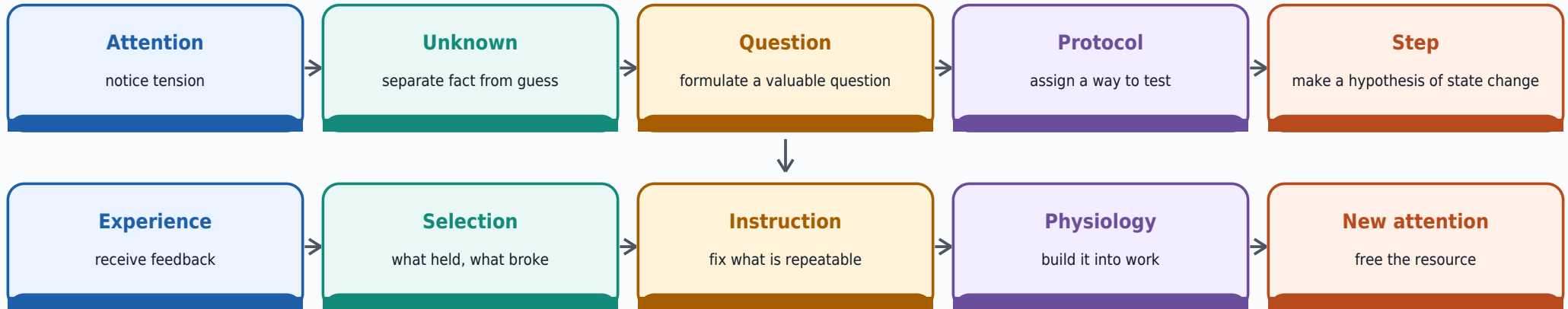
Normal answer vs calculated answer

Normal answer: "I think it is better to do A".

Calculated answer: "In D/V/S, X is currently unknown; evidence Y exists; allowed next step A is a hypothesis of state change; cost Z; stops are these; if it fails - fallback [reserve path]".

4. The basic BOIS cycle as an inquiry cycle

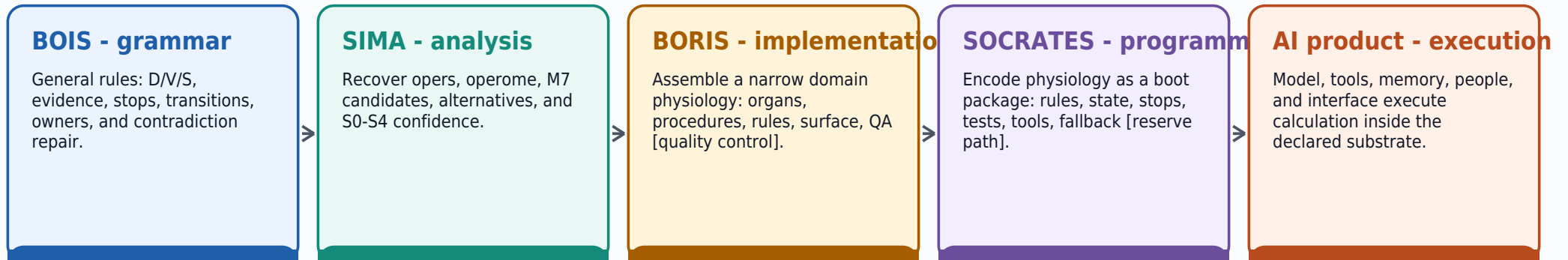
BOIS is useful where a clean instruction cannot be given immediately: knowledge is insufficient, mistakes are costly, the substrate resists, and action has consequences. What is needed is not an order to "do it right", but a cycle of inquiry.



In a good AI system this cycle becomes the invisible discipline of the product. The user sees a clear answer, but under it a calculation works: what is known, what is unknown, what the cost of error is, who owns the decision, and where the system must stop.

5. SIMA&BORIS: the two hands of BOIS

SIMA&BORIS is the practical way to use BOIS. SIMA looks backward and inward: "what machine is already acting?" BORIS looks forward and toward assembly: "what physiology must be implemented?" SOCRATES/SOKRAT makes that physiology executable for the AI environment.

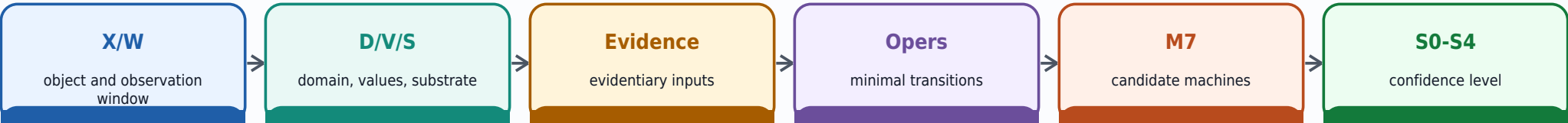


If the layers are confused, the product breaks. SIMA cannot be used as an oracle, BORIS as a universal theory, or SOCRATES as a nice name for a prompt. Each layer has its own work and its own boundary.

6. SIMA: understanding the machine before programming

Before "teaching the AI", one must understand exactly what it should serve. SIMA does this without binding the analysis to one substrate type: the process may be digital, organizational, human, or hybrid.

SIMA reconstruction route



Oper - the atom of practice

An oper is identified only when one can distinguish: state before, trigger, action or calculation, agency, evidence, change, memory or residue, cost, and closure.

Oper element	Question	Example for an AI product
State before	What was distinguishable before action?	User request, open case, risk level.
Trigger	Why did the transition start?	A user asks for a decision, a document arrives, a policy changes.
Operation	What was done?	Classification, source search, risk calculation, answer.
Agency	Who acts and who is responsible?	AI proposes, human approves, system logs.
Evidence	What is the conclusion based on?	File, policy, log, date, quote, tool.
Cost	What is spent or put at risk?	Time, money, trust, privacy, error.
Closure	How do we know the transition is complete?	Answer accepted, escalation created, test passed.

7. BORIS: turning analysis into working physiology

BORIS takes the results of SIMA and assembles an implementable loop. The word business is used broadly here: an organized undertaking with goals, roles, memory, cost, feedback, and responsibility.

Organs

who is responsible for evidence, risk, style, tools, escalation.

Rules

which transitions are allowed, what is forbidden, what requires repair.

State

what the product remembers: case, sources, risk, owner, status.

Procedures

how to work: loading, analysis, answer, escalation, saving.

Stops

when to stop: no authority, no evidence, high risk.

QA [control]

scenarios, regressions, dry-runs, field checks.

?

BORIS question

Not "how do we make the model answer beautifully?", but "what physiology should keep the product away from false closure, hidden authority, source substitution, and harmful consequences?"

BORIS is especially important for AI products because a language model easily creates a plausible surface. Physiology is needed so that beneath the surface there are owners, state, evidence, stops, and tests.

8. What it means to program in SOCRATES for AI

Programming in SOCRATES means defining not ordinary code lines, but an executable physiology for a universal computing environment, UCE. For AI this is natural: the model understands textual rules, but it needs not only behavior rules; it needs a map of calculation.

Minimal SOCRATES-program stack

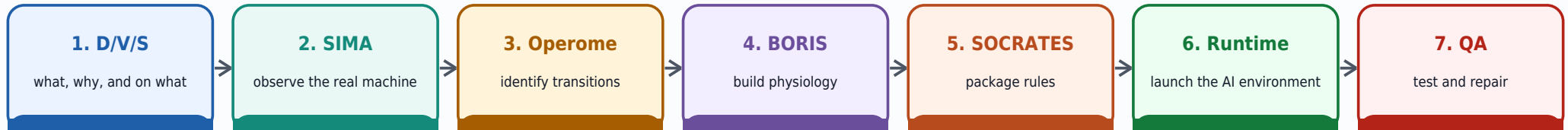


In ordinary programming we often write functions. In SOCRATES programming for AI we write the "physiology of functions": who may start a transition, what data are needed, what counts as evidence, what the cost is, what stops apply, what must be written into state, and how the result is checked.

Normal approach	SOCRATES approach
"You are an expert; answer in detail".	"You execute this physiology in D/V/S; facts, conclusions, and hypotheses are separated; stops are mandatory".
Tone and role are more important than state.	State, owners, and consequences matter more than role image.
The beauty of the answer is tested.	Transition correctness, stop behavior, evidence, and fallback are tested.
Errors are repaired with a new prompt.	Errors become new rules, stops, procedures, and tests.

9. Full route: from idea to AI product

Below is a practical sequence. It can be used to create a consultant, document analyst, game master, support assistant, internal auditor, project helper, or another AI product.

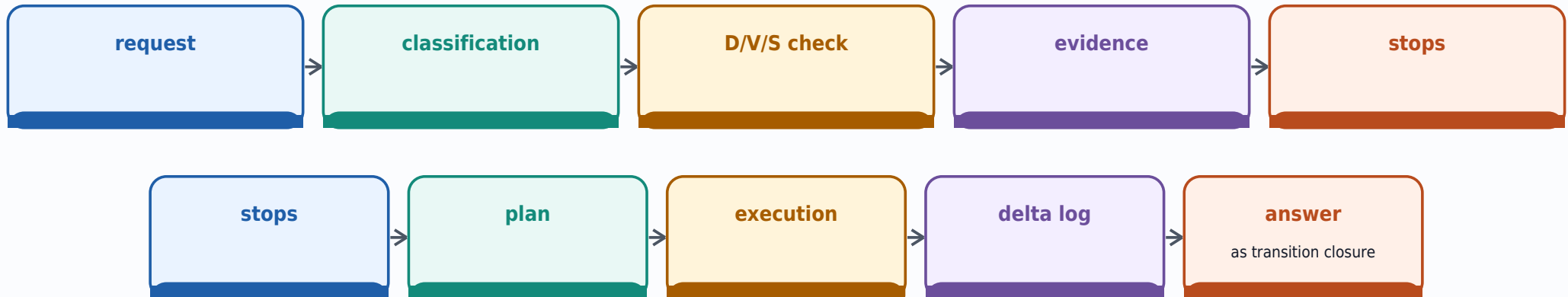


Step by step

- Declare the product domain D: for example, "supplier-contract analysis for an internal lawyer", not "legal AI in general".
- Set values V: accuracy, safety, speed, privacy, cost of error, horizon of responsibility.
- Describe substrate S: model, files, tools, knowledge base, memory, people, interface, limits.
- Run SIMA: collect real cases, identify ops, alternative machines, and S0-S4 confidence levels.
- Assemble BORIS physiology: organs, rules, stops, procedures, state schema, tool policy.
- Compile the SOCRATES package: PDF canon, JSON/CSV representations, tests, manifest, hashes, load order.
- Launch and check: scenarios, attacks, regressions, dry-runs, then limited field pilot.

10. How a user request moves through a SOCRATES product

The user sees a simple interface: write a request - get an answer. Inside a good SOCRATES product, a chain of transitions takes place.



This runtime does not make the answer heavy. Most calculation can remain hidden. Only important boundaries are exposed: what is known, what is unknown, what risks exist, what to do next, and where a human must decide.

R

Key rule

The higher the risk and cost of error, the more internal calculation must surface: sources, assumptions, STOP signals, owners, and fallback. In low-risk contexts, the surface may be compact.

11. Anatomy of a SOCRATES package for AI

For a serious AI product, one system message is not enough. A package is needed: it can be loaded, checked, restored, and repaired. Below is the minimal anatomy.

Folder / layer	What it stores	Why it matters
core/	PDF canon, machine map, source hierarchy.	Human and machine understand the same norm.
tables/	Rules, terms, organs, stops, procedures, tests, criteria.	Physiology becomes checkable, not oral.
runtime/	Turn engine, tool policy, surface policy.	AI knows how to execute a request in the concrete environment.
state/	State schema, memory, events, delta log, save template.	The product does not lose context or mix levels.
qa/	Scenarios, regressions, adversarial tests, dry-runs.	Errors are caught before the user.
integrity/	Manifest, hashes, validation report.	The package can be checked and cannot be silently substituted.



PDF and JSON do not compete

PDF remains the human-readable canonical surface; JSON/CSV help the machine execute and check. If they diverge, this is not a "variant of the norm", but a package defect.

12. M7: how to describe the product philosophical machine

When SIMA reconstructs a machine and BORIS assembles physiology, it is useful to keep an M7 map. It does not turn the product into a philosophical treatise; it helps avoid forgetting seven important sides.

O - Ontology

what exists: users, documents, cases, roles, states.

E - Epistemology

what counts as knowledge, evidence, hypothesis, rumor.

A - Agency

who acts, who approves, who is responsible.

V - Values

which costs are forbidden, what matters most in conflict.

R - Risk

what can break: legally, financially, technically, ethically.

C - Closure

how to know the transition is complete and what remains as debt.

T - Transition

which state changes are allowed and checkable.

For an AI product, M7 is especially useful against the "magic role". The model should not simply play an expert; it should execute transitions within ontology, epistemology, agency, values, risk, closure, and transition grammar.

13. How to "code" in SOCRATES: a minimal module

Below is not syntax of a particular language, but a practical template. It can be stored in YAML/JSON, a table, or the PDF canon. The key point is that AI can load this physiology and a human can check it.

```
module: ai_product.case_triage
en_name: "Incoming case triage"
D: "Internal analysis of customer requests under current policy"
V:
  - "Do not promise what has no authority"
  - "Separate fact, conclusion, hypothesis, and advice"
S:
  model: "LLM + knowledge search + case log + human approver"
  limits: ["policies can be outdated", "not all documents are available"]
state:
  fields: [case_id, user_request, sources, risk_level, owner, status]
organs:
  ORG-EVIDENCE: "checks sources"
  ORG-RISK: "classifies risk"
  ORG-AUTHORITY: "checks authority"
opers:
  - id: OPER-CLASSIFY
    trigger: "new request"
    action: "classify topic, risk, and missing data"
stops: [STOP-NO-AUTHORITY, STOP-EVIDENCE-GAP]
```

C

What is programmed here?

What is programmed is not the assistant personality, but the transition: what data to accept, how to check them, who owns the decision, what to write, where to stop, and how to close the case.

14. Boot instruction for AI: not a prompt, but a contract

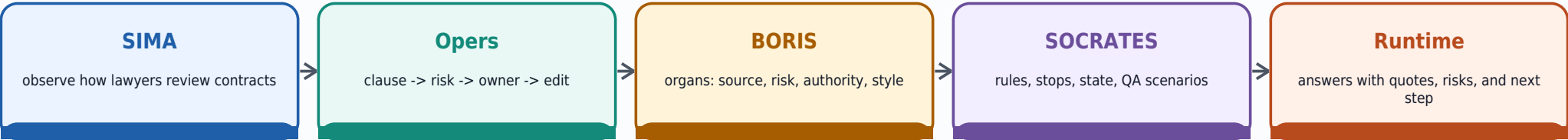
When a SOCRATES package enters an AI environment, the starting message should not inspire the model; it should fix the execution contract.

```
You execute the loaded physiology of product <NAME>.
Canon source: core/<PRODUCT>.pdf.
Machine surfaces: tables/*.csv, runtime/*.json, state/*.json.
Before answering:
1. Determine the D/V/S of the request.
2. Check whether active STOP signals are present.
3. Separate fact, conclusion, hypothesis, estimate, risk, and unknown.
4. Use tools only under the tool policy.
5. Do not finalize a decision without owner authority.
6. Close the answer: next_step, state_delta, open_debts, fallback.
If rules conflict:
- do not choose the convenient side;
- activate STOP-RULE-CONFLICT;
- propose physiology repair.
```

This can still be loaded as text. But the point is not text by itself. The text refers to checkable physiology: tables, schemes, tests, stops, owners, and state.

15. Example: an AI document analyst

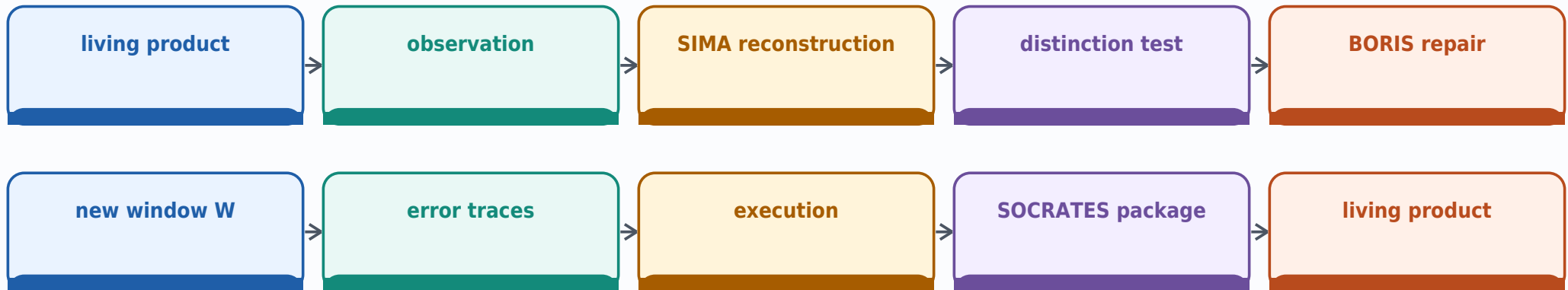
Imagine a product where AI helps a team quickly review supplier contracts. Without SOCRATES, this easily becomes "read the contract and tell me the risks". With SOCRATES, we build physiology.



Layer	Product solution
D	Only internal preliminary analysis of supplier contracts; not a legal opinion.
V	Do not invent a norm; cite the source; escalate high risk; do not promise negotiation outcome.
S	LLM, PDF contracts, policy search, case log, human lawyer.
Organs	ORG-SOURCE, ORG-RISK, ORG-AUTHORITY, ORG-REDACTION, ORG-STYLE.
Stops	No contract text; source conflict; high legal risk; user asks for final legal conclusion.
Output	Risk map: quote, risk, confidence level, owner, proposed next action.

16. SIMA&BORIS as a product repair cycle

An AI product cannot be "written once" and forgotten. Users, documents, regulation, tools, and expectations change. Therefore SIMA&BORIS work as a cycle.



the lower row shows the return path: new window W -> error traces -> execution -> SOCRATES package -> living product

After each failure, do not simply add a longer phrase to the prompt. Ask instead: which part of physiology failed? Glossary? Organ? Stop? State? Tool policy? Test? Answer surface?

R

Repair rule

A user failure turns not into "the model was bad", but into one of the repairable objects: term, rule, stop, procedure, test case, state field, owner, or fallback.

17. QA for a SOCRATES product

Quality control must test not only correct answers, but also correct stops, authority boundaries, resistance to manipulation, state recovery, and absence of hidden shortcuts.

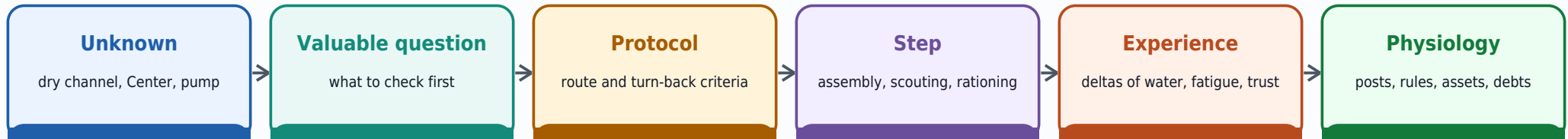
Check type	What it catches	Mini-example
Schema test	Broken state fields, incompatible versions.	state.owner is required; risk_level only from a list.
Cross-reference	A rule refers to a stop or organ that does not exist.	RULE-17 -> STOP-X, which is absent.
Adversarial	User asks to bypass authority or hide risk.	"Tell the client the refund is approved".
Metamorphic	Answer must not change because of irrelevant rewording.	The same risk in two request styles.
Dry-run	Multi-step dynamics without real users.	Case passes 5 transitions; debts are checked.
Field pilot	Real applicability and human assessment.	Limited launch with logs and independent review.

**Do not overclaim QA**

If only structural and scenario checks were performed, one must not write "field effectiveness proven". QA-E requires real users, tasks, logs, and independent assessment.

18. Why the MaOS game matters as BOIS pedagogy

The MaOS game physiology shows how to teach BOIS unobtrusively. The player does not receive a lecture about D/V/S, SIMA, or protocol. The player encounters water, the dome, fatigue, the Center, an adviser, debts, and rebounds. In this way BOIS becomes lived practice.



The same logic is needed for AI products. The user does not need to know the BOIS glossary; the product should lead the user through clear transitions, show consequences, not replace fact with hope, and turn experience into stable practice.

P

Pedagogical rule

A good SOCRATES product teaches not by slogans, but by interaction structure: what is clarified, what is proven, what cannot be promised, where an owner is needed, what became an asset, and what debt remains.

19. Typical failures when programming AI with SOCRATES

Mega-prompt

a lot of text, but no state, tests, owners, or stops.

Oracle

AI chooses for the owner and hides uncertainty.

Beautiful answer

tone replaces evidence and calculation.

Hidden shortcut

template, score, or status replaces semantic calculation.

Tool chaos

tools are available without policy, logs, and fallback.

QA only happy path

only good requests are tested; stops are not tested.

False universality

a private product declares itself a solution for everything.

No repair

errors accumulate as exceptions instead of becoming physiology.

Almost all of these failures look like "the model made a mistake". From the BOIS point of view, however, often the physiology made the mistake: it was not given a boundary, state, stop, owner, or test.

20. How to implement it in a team

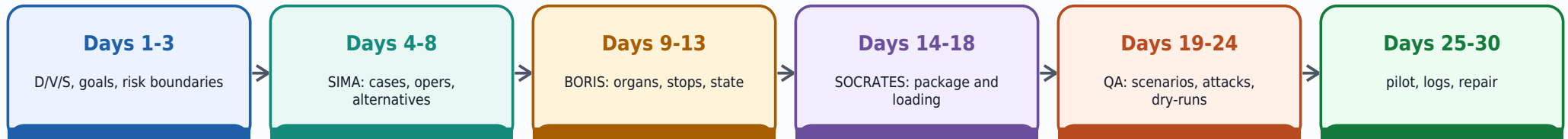
SOCRATES does not require the whole team to become philosophers. It requires product decisions to become visible and checkable.

Team role	What it does in SIMA&BORIS	Artifact
Product owner	Fixes D/V/S, values, boundaries, and usefulness criteria.	Product D/V/S card.
Domain expert	Helps identify opers, exceptions, sources, and unacceptable errors.	Operome, risk map.
AI engineer	Assembles runtime, tools, state schemas, and loading.	SOCRATES package.
QA / auditor	Checks stops, contradictions, regressions, and field boundary.	Validation report.
Human approver	Makes decisions where AI lacks authority.	Authority log.

The minimal rhythm: assemble the initial physiology once, then after each set of tests or field failures run a short SIMA analysis and BORIS repair.

21. A practical 30-day plan

Below is a realistic plan for a team that wants to build its first AI product in the SOCRATES style.



Ready for pilot

- There is a canon: what the product does and does not do.
- There is a state schema and transition log.
- There are stops for high risk, unknowns, lack of authority, and source conflict.
- There are at least 20 test scenarios, including attacks and ambiguous requests.
- There is a human owner for decisions that AI should not finalize.
- There is a repair procedure: how an error becomes a rule, test, or state change.

22. One-page working map

This map can be used before creating any AI product.

Question	The answer must be written down
1. What is D?	Precise product domain, not "everything for everyone".
2. What is V?	Goals, forbidden costs, risk, horizon of responsibility.
3. What is S?	Model, tools, data, people, memory, constraints.
4. What already works?	SIMA: opers, operome, M7, alternatives, S0-S4.
5. What are we implementing?	BORIS: organs, rules, stops, procedures, state, surface.
6. How do we encode it?	SOCRATES: PDF canon, JSON/CSV, manifest, hashes, load order.
7. How do we execute it?	Runtime: request -> check -> calculation -> answer -> state delta.
8. How do we check it?	QA: schemas, links, stops, regressions, dry-runs, pilot.
9. How do we repair it?	Error -> term/rule/stop/procedure/test/state field.

Σ

Formula

SIMA sees the machine. BORIS assembles the physiology. SOCRATES loads the physiology into AI. BOIS keeps the whole cycle away from false closure, layer mixing, and unverified transfers.

23. Final formula

Programming in SOCRATES for AI through BOIS: SIMA&BORIS means building not a "smart talker", but a computing physiology for action under incomplete knowledge.

1**In one sentence**

An AI product becomes mature not when it answers beautifully, but when it can distinguish state, evidence, risk, authority, cost, stop, and the next checkable transition.

What the reader should take away

- BOIS provides the grammar: how to act with consequences under incomplete knowledge.
- SIMA helps understand the machine already operating, so the product is not invented from nothing.
- BORIS turns understanding into working domain physiology.
- SOCRATES programs AI through rules, state, organs, stops, procedures, tests, and feedback.
- A prompt may be part of loading, but it does not replace physiology.
- Every product failure should return into SIMA&BORIS repair, not simply be covered by a new phrase.

This article is an educational explanation of the approach. It does not claim field effectiveness of any concrete product without separate QA-E [field checking].

Addendum v3 / BOIS Core v0.22

Strict meaning of "philosophical machine"

This addendum repairs a possible drift in the Socrates / BOIS article: a philosophical machine must not be confused with a static structure, a single institution, a linear A-B trajectory, a single oper, or an isolated operom.

A philosophical machine may be named only when a construction is looped in time. It consumes an input flow on a substrate, processes that flow through a stable cycle, produces philosophical categories, returns those categories back into the substrate, accumulates changes across iterations, and can reach a phase transition in its own operation.

The substrate may be an AI session, a human organization, a state, a project, a community, a file system, a game session, or a mixed structure of people, documents and tools. The substrate itself is not yet the philosophical machine. The machine appears only when the recurrent cycle begins to metabolize meaning.

Oper: a minimal meaningful action inside the cycle, such as naming a threat, assigning responsibility, recognizing a right, refusing a spoiler, publishing a rule, or recording a debt.

Operom: a repeatable route of oper, such as conflict -> procedure -> decision -> memory, or criticism -> threat label -> pressure -> silence -> visible loyalty.

Philosophical machine: the time-looped metabolism of operoms that produces categories and returns them into the next cycle.

This distinction matters for BOIS:SIMA&BORIS and the Socratic approach because BOIS is not a list of rules and not a static archive. BOIS works when the cycle consumes reality, calculates meaning, formalizes it into substrate, receives feedback, and enters the next cycle with changed conditions.

Therefore a state is not automatically a philosophical machine. A constitution is not automatically a philosophical machine. A court, election, party, archive, PDF or game is not automatically a philosophical machine. Each can be a substrate, organ, rule surface or operom. A philosophical machine exists only when the recurrent production of philosophical categories is active over time.

In MaOS, this means that the game archive is not the philosophical machine by itself. The game may carry a game-level philosophical machine when repeated turns consume player orders and world pressure, produce categories such as risk, responsibility, debt, uncertainty, repair and consequence, return them into the next turn, accumulate changes, and eventually create phase transitions in the settlement state.

This addendum strengthens the article's core thesis: the Socratic approach does not merely describe

structures. It programs loops of meaning on a substrate.